

Operating Systems

CSC401



SK Hafizul Islam

Department of CSE, IIIT Kalyani

April 15 & 16, 2026

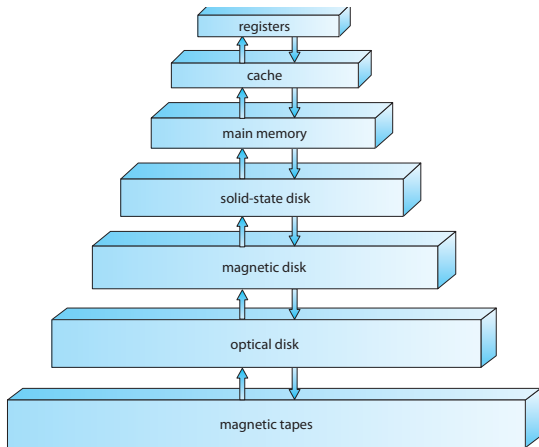


1 Mass-Storage Structure

- Magnetic Disk
- Disk Structure
- Disk Scheduling
 - FCFS Scheduling
 - SSTF Scheduling
 - SCAN Scheduling
 - C-SCAN Scheduling
 - LOOK Scheduling
 - C-LOOK Scheduling

2 Selecting a Disk-Scheduling Algorithm

3 Suggested Readings



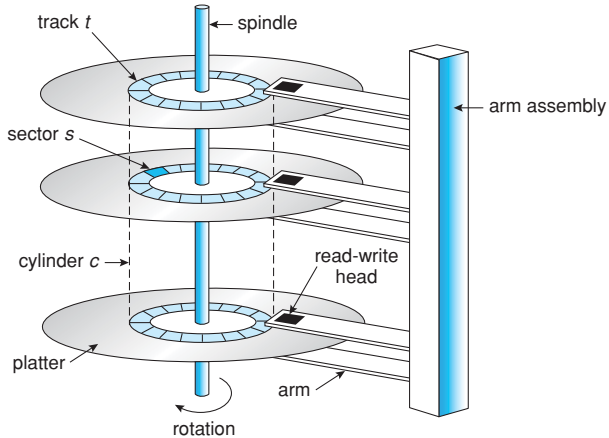
- **Main Memory/RAM (volatile)**

- ☞ implemented using dynamic random-access memory (DRAM) semiconductor technology.

- **Secondary storage**

- ☞ Magnetic disk - storage for both programs and data.
- ☞ Solid-state disk - faster than magnetic disks and are nonvolatile.
- ☞ Flash memory - slower than DRAM but needs no power to retain its contents.

Magnetic Disk





- Provide bulk of secondary storage of modern computers

- ☞ Drives rotate at 60 to 250 times per second.
- ☞ Platter diameters range from 1.8 to 3.5 inches.
- ☞ **Transfer rate** is rate at which data flow between drive and main memory.
- ☞ **Positioning time (random-access time) = seek time + rotational latency.**
 - ☞ **Seek time** is defined as the time required by the read/write head to move from one track to another.
 - ☞ **Rotational latency** is the time required by the read/write head to rotate to the requested sector from the current sector.
- ☞ **Head crash** results from disk head making contact with the disk surface.
 - ☞ That's bad
 - ☞ Disks can be removable
 - ☞ cannot be repaired - replace the disk



- Drive attached to computer via I/O bus

☞ Busses vary

- ☞ Enhanced Integrated Drive Electronics (EIDE),
- ☞ Advanced Technology Attachment (ATA),
- ☞ Serial ATA (SATA),
- ☞ External SATA (eSATA),
- ☞ Universal Serial Bus (USB),
- ☞ Fiber Channel (FC), and
- ☞ Small Computer System Interface (SCSI).

☞ The data transfers on a bus are carried out by special electronic processors called **controllers**.

☞ **Host controller** in computer uses bus to talk to **disk controller** built into drive or storage array.



- **Seek time** is the time required to move the disk arm to the required track.
 - ➡ It consists of (1) initial startup time (2) the time taken to traverse the tracks.
 - ➡ the traversal time is not a linear function of the number of tracks but includes a settling time (time after positioning the head over the target track until track identification is confirmed).
 - ➡ $T_s = M \times N + S$
 - ➡ T_s = seek time
 - ➡ N = number of tracks to traverse
 - ➡ M = constant depends on the disk drive
 - ➡ S = startup time
- **Rotational delay** is the time required for the addressed area of the disk to rotate into a position where it is accessible by the read/write head.



- **Transfer time** is time to or from the disk depends on the rotation speed of the disk in the following fashion:
 - $T = \frac{b}{r \times N}$
 - $T \Rightarrow$ Transfer time,
 - $b \Rightarrow$ number of bytes to be transferred,
 - $N \Rightarrow$ number of bytes on a track, and
 - $r \Rightarrow$ rotation speed, in revolutions per second.
- **Average access time:** $T_a = T_s + \frac{1}{2r} + \frac{b}{r \times N}$



- Consider a disk pack with a seek time of 4 ms and rotational speed of 10,000 rotations per minute (RPM). It has 600 sectors per track and each sector can store 512 bytes of data. Consider a file stored in the disk. The file contains 2,000 sectors. Assume that every sector access necessitates a seek, and the average rotational latency for accessing each sector is half of the time for one complete rotation. What is the total time (in ms) needed to read the entire file?

☞ $T_s = \text{Seek time} = 4\text{ms}$

☞ $r = \text{One rotation is performed in } 60/10000 = 0.006 \text{ sec} = 6\text{ms}$

☞ Rotation delay $T_r = 6/2 = 3\text{ms}$

☞ One rotation reads 600 sectors and time per rotation = 6 ms

☞ Transfer time per sector = $6 / 600 = 0.01 \text{ ms}$.

☞ Total time per sector = Seek time + Rotation delay + Transfer time = $(4 + 3 + 0.01) = 7.01 \text{ ms}$

☞ Total time for 2000 sectors (to read entire file) = $2000 \times 7.01 = 14020 \text{ ms} = 14020 \text{ ms}$



- Disk drives are addressed as large one-dimensional arrays of logical blocks, where the logical block is the smallest unit of transfer.
- The one-dimensional array of logical blocks is mapped into the sectors of the disk sequentially.
 - Sector 0 is the first sector of the first track on the outermost cylinder
 - Mapping proceeds in order through that track, then the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost.
 - Logical to physical address should be easy
 - Except for bad sectors
 - Non-constant number of sectors per track via constant angular velocity



- One of the responsibilities of the OS is to use the hardware efficiently.
 - ☞ for the disk drives - OS must ensure fast access time and high data transfer rate.
 - ☞ minimize seek time
 - ☞ Seek time $\approx$ seek distance
- We can improve both the access time and data transfer rate (bandwidth) by scheduling the servicing of disk I/O requests in a good order
 - ☞ Disk scheduling algorithm.
- Whenever a process needs I/O to or from the disk, it issues a system call to OS. The request specifies several pieces of information:
 - ☞ Whether this operation is input or output
 - ☞ What the disk address for the transfer is
 - ☞ What the memory address for the transfer is
 - ☞ What the number of sectors (total bytes) to be transferred is



- OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queue
 - 👉 Optimization algorithms only make sense when a queue exists
- Note that disk controllers have small buffers and can manage a queue of I/O requests (of varying “queue depth”)
- Several algorithms exist to schedule the servicing of disk I/O requests

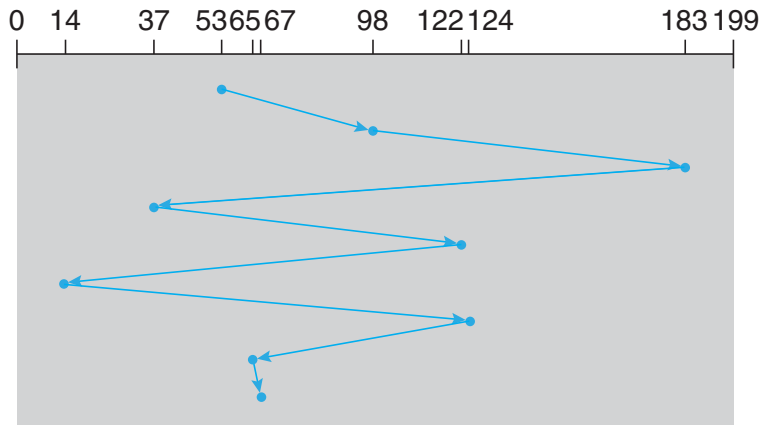


- FCFS
- SSTF
- SCAN
- C-SCAN
- LOOK
- C-LOOK



- Consider, for example, a disk queue with requests for I/O to blocks on cylinders 98, 183, 37, 122, 14, 124, 65, 67, in that order.
- Initially, the disk head is at cylinder 53.

FCFS Scheduling



- Total Head Movement:
$$= |98-53| + |183-98| + |37-183| + |122-37| + |14-122| + |124-14| + |65-124| + |67-65|$$
$$= 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2$$
$$= 640 \text{ cylinders}$$



- **Advantages:**

- ✎ Easy to implement and understand.
- ✎ No starvation
- ✎ Easy to predict performance and head movement since it's sequential.

- **Disadvantages:**

- ✎ Can result in high average seek time and long waiting times, especially if requests are scattered across the disk.

Shortest-Seek-Time-First (SSTF) Scheduling



- A disk queue with requests for I/O to blocks on cylinders 98, 183, 37, 122, 14, 124, 65, 67, in that order.
- Initially, the disk head is initially at cylinder 53.
- SSTF selects the request with the minimum seek time from the current head position, i.e., chooses the pending request closest to the current head position.

👉 $|53 - 65| = 12$ (1)

👉 $|53 - 67| = 14$ (2)

👉 $|53 - 37| = 16$ (3)

👉 $|53 - 14| = 39$ (4)

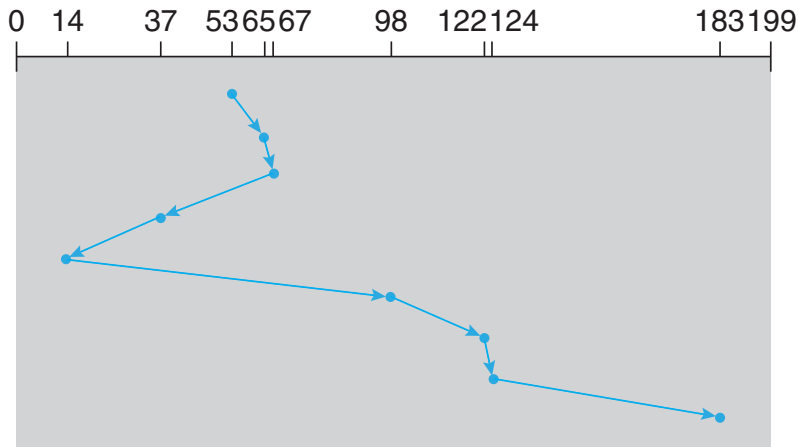
👉 $|53 - 98| = 45$ (5)

👉 $|53 - 122| = 69$ (6)

👉 $|53 - 124| = 71$ (7)

👉 $|53 - 183| = 130$ (8)

Shortest-Seek-Time-First (SSTF) Scheduling



- Total head movement of 236 cylinders.



- **Advantages:**

- ✎ Minimizes Average Seek Time - Always picks the closest request, leading to better performance than FCFS or SCAN in many cases but not always.

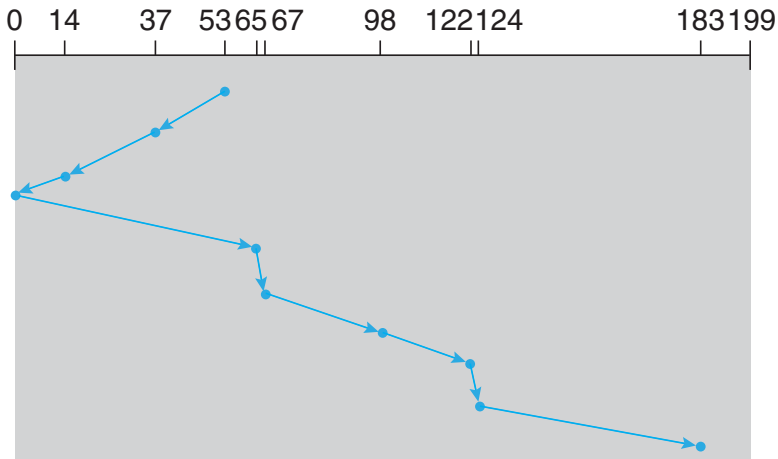
- **Disadvantages:**

- ✎ It is not optimal.
- ✎ SSTF scheduling (**Greedy algorithm**) is a form of SJF scheduling - **may cause starvation** of some requests
- ✎ We can do better by moving the head from 53 to 37, even though the latter is not closest, and then to 14, before turning around to service 65, 67, 98, 122, 124, and 183. This strategy reduces the total head movement to 208 cylinders.



- A disk queue with requests for I/O to blocks on cylinders 98, 183, 37, 122, 14, 124, 65, 67, in that order. Initially, the disk head is initially at cylinder 53.
- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues. SCAN algorithm sometimes called the **elevator algorithm**.
 - 👉 OS may define a default system policy - Start toward higher tracks or lower tracks.

SCAN Scheduling



- Total head movement of 208 cylinders.



- **Advantages:**

- ☞ Reduces the average seek time compared to FCFS and SSTF.
- ☞ Every request gets serviced in a predictable order - no starvation.
- ☞ Efficient for Heavy Loads - Performs well when many requests are pending.

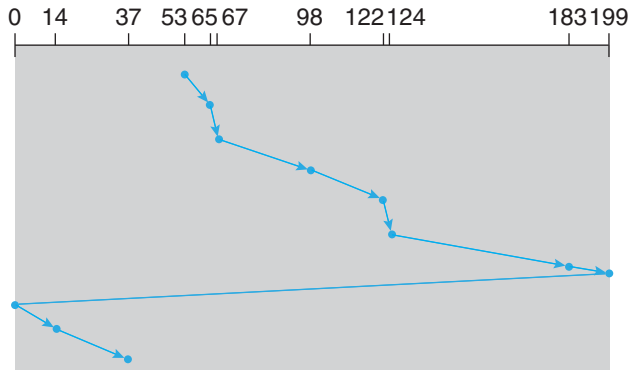
- **Disadvantages:**

- ☞ If a request arrives in the queue just in front of the head, it will be serviced almost immediately; a request arriving just behind the head will have to wait until the arm moves to the end of the disk, reverses direction, and comes back.
- ☞ If the requests are uniformly dense, largest density at other end of disk and those wait the longest.
- ☞ Slightly more complex than FCFS and SSTF due to the need to manage direction and order.
- ☞ Head always goes to the end even if no requests exist there - causes extra seek time.



- Provides a more uniform wait time than SCAN.
- The head moves from one end (typically outward $\Rightarrow$ higher-numbered cylinders) of the disk to the other, servicing requests as it goes.
 - ☞ When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip.
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one.

C-SCAN Scheduling



- Total head movement of $|53 - 65| + |65 - 67| + |67 - 98| + |98 - 122| + |122 - 124| + |124 - 183| + |183 - 199| + |199 - 0| + |0 - 14| + |14 - 37| = 12 + 2 + 31 + 24 + 61 + 16 + 199 + 14 + 23 = 382$ cylinders.



- **Advantages:**

- ☞ Uniform Wait Time - Provides more uniform service than SCAN by treating requests equally regardless of position.
- ☞ No Starvation - Every request will eventually be serviced as the head sweeps the disk regularly.

- **Disadvantages:**

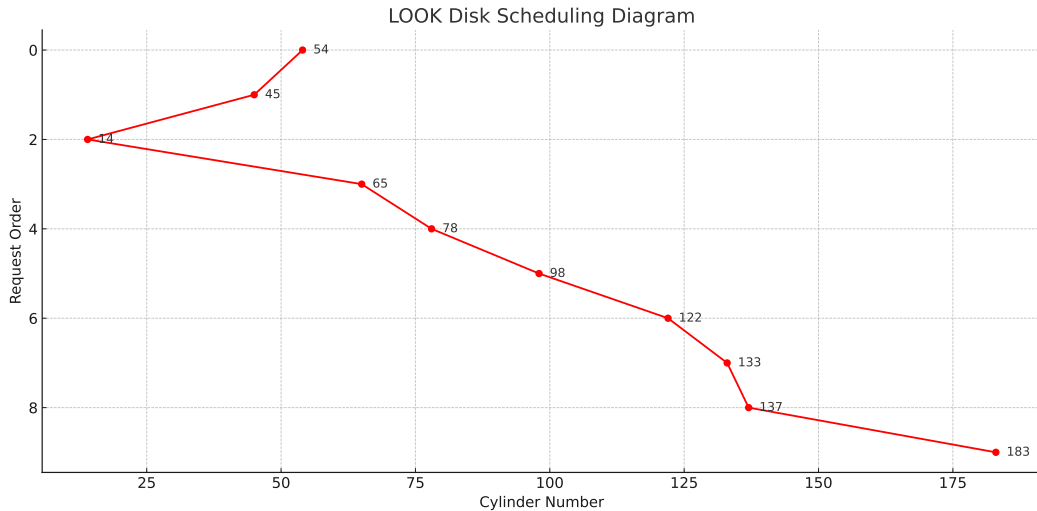
- ☞ The head has to return from the end to the beginning without servicing requests, which adds extra movement.
- ☞ May involve more travel compared to LOOK or SSTF in certain conditions.
- ☞ More Complex than FCFS or SSTF - Needs additional logic to handle the circular movement.



- LOOK is an improved version of the SCAN algorithm. It's more efficient because it “**looks**” ahead to see if there are any requests before moving to the end of the disk.
 - ➡ the disk arm moves in one direction (just like SCAN).
 - ➡ instead of going all the way to the end of the disk, it stops at the last request in that direction.
 - ➡ then it reverses direction and starts serving requests the other way.
- So, it “**looks**” for pending requests before moving further.



- Consider the following disk request sequence for a disk with 200 tracks 98, 137, 122, 183, 14, 133, 65, 78. Initially, the disk head is at cylinder 54, and moving in left direction. Find the number of head movements in cylinders using LOOK scheduling.
- The total number of head movements using the LOOK scheduling algorithm is 209 cylinders.





- **Advantages:**

- Avoids unnecessary movement to disk ends
- Does not change direction frequently
- Better performance than FCFS and SCAN
- No starvation

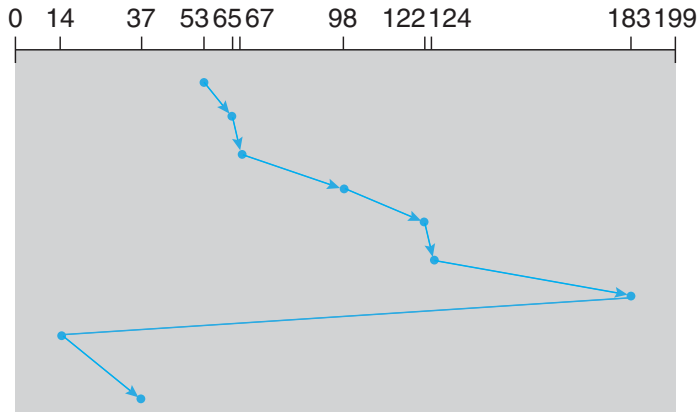
- **Disadvantages:**

- Non-uniform waiting time - Requests at one end may wait longer
- Does not always minimize seek time - in certain conditions (sparse or uneven requests), direction changes may increase seek time because the disk head may travel a short distance, reverse direction, and then move again, leading to additional head movement.



- C-LOOK is an enhanced version of the C-SCAN algorithm.
- Arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk.

C-LOOK Scheduling



- Total head movement of $|53 - 65| + |65 - 67| + |67 - 98| + |98 - 122| + |122 - 124| + |124 - 183| + |183 - 14| + |14 - 37| = 12 + 2 + 31 + 24 + 61 + 16 + 169 + 23 = 338$ cylinders.



- **Advantages:**

- ☞ Skips unnecessary movement to disk ends (unlike C-SCAN), reducing total seek time.
- ☞ Every request is guaranteed service on each circular pass - no starvation.
- ☞ Improved performance over C-SCAN.

- **Disadvantages:**

- ☞ The jump from the last to the first request in the next cycle adds some overhead.
- ☞ For a small number of requests, simpler algorithms (like SSTF) might perform better.
- ☞ Requires managing sorted queues and circular logic.



- SSTF is common and has a natural appeal because it increases performance over FCFS.
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk, because they are less likely to cause a starvation problem.
- Performance depends on the number and types of requests.
 - ☞ If the queue has just one outstanding request, all behaves FCFS.
- Requests for disk service can be influenced by the file-allocation.
 - ☞ Contiguously allocated file will generate several requests that are close together on the disk, resulting in limited head movement.
 - ☞ A linked or indexed file may include blocks that are widely scattered on the disk, resulting in greater head movement.



- The location of directories and index blocks is also important.
 - ☞ Every file must be opened to be used, and opening a file requires searching the directory structure.
 - ☞ Caching the directories and index blocks in main memory can also help to reduce the disk-arm movement.
- Due to these complexities, the disk-scheduling algorithm should be written as a separate module of the OS, allowing it to be replaced with a different algorithm if necessary
- Either SSTF or LOOK is a reasonable choice for the default algorithm



- **Chapter 10**, A. Silberschatz, P.B. Galvin, and G. Gagne, “**Operating System Concepts**” 9th Ed., Wiley Publishers, 2016.
- **Chapter 11**, W. Stallings, “**Operating Systems Internals and Design Principles**”, 9th Ed, Pearson, 2018.

Thank You